

PROJECT 1

Adaptive 3D Gaussian Splatting for Real-Time Online RGB-D Reconstruction

Advanced Machine Learning Core + AI Systems Optimization Layer

Version 3.0 - Research-focused design + repository implementation audit (2026-08-28)

Baseline: 3D Gaussian Splatting (3DGS) + RTG-SLAM (SIGGRAPH 2024)

Primary research theme: Gaussian-level utility estimation and budget-constrained scheduling for online RGB-D reconstruction.

Scope rule: ML/algorithmic contribution first; CUDA is the implementation vehicle for measured speedups; viewer polish is not a research milestone.

1. Executive Summary

Mục tiêu của project là xây dựng một pipeline 3D Gaussian Splatting (3DGS) cho online RGB-D reconstruction, trong đó contribution nghiên cứu tập trung vào bài toán: với một compute budget hữu hạn ở mỗi frame, Gaussian nào nên được optimize, Gaussian nào nên được densify và Gaussian nào nên được prune để tối đa hóa chất lượng reconstruction.

3DGS/RTG-SLAM đã giải quyết nhiều thành phần nền tảng như Gaussian representation, differentiable splatting, RGB-D online optimization, selective optimization và compact mapping. Vì vậy, project này không claim những thành phần đó là novelty. Chúng được giữ làm nền tảng và baseline để cô lập contribution mới.

Contribution trung tâm được định nghĩa lại thành ba lớp liên kết: (1) Gaussian utility estimation, (2) budget-constrained scheduling, và (3) closed-loop adaptation dựa trên quality/latency/memory feedback. Ý tưởng continuous importance, adaptive thresholds và error-driven densification được giữ lại nhưng trở thành thành phần của framework hoặc ablation, không phải các contribution ngang hàng.

Layer	Mục tiêu	Thành phần	Trạng thái
Research core	Ước lượng utility của từng Gaussian	Gaussian state, influence, uncertainty, temporal statistics, utility model	Bắt buộc
Adaptive policy	Phân bổ compute dưới constraint	optimization scheduler, densification policy, pruning policy	Bắt buộc
Online reconstruction	Cập nhật scene từ RGB-D stream	tracking, render, RGB-D loss, map update	Bắt buộc
AI Systems	Đạt target latency với cùng quality	CUDA kernels, memory layout, profiling, scheduling runtime	Phát triển sau baseline
Viewer	Demo end-to-end	OpenGL viewer, camera control	Không phải contribution chính

2. Current Repository Implementation Audit (2026-08-28)

The repository is substantially beyond a paper-only design: it contains a working research prototype, unit/gradient tests, dataset loaders, attribution, adaptive scheduling, CUDA kernels, experiments, and benchmark artifacts. However, several claims in the earlier roadmap must be downgraded from “achieved” to “prototype / not yet validated” until matched-budget and end-to-end evidence is available.

2.1. What is actually implemented

- Gaussian representation, projection, rasterization, surface-aware depth rendering, RGB-D losses, densification, pruning, per-Gaussian attribution, importance estimation, and policy scheduling are implemented as separate research modules.
- The repository contains a dedicated attribution module that maps pixel-level color/depth error back to the top-K Gaussian contributors instead of assigning every Gaussian the global mean error.
- The scheduler exposes FULL, RANDOM, BINARY, TOP_K, and BUDGET_AWARE policies and includes an importance/cost greedy selection path plus profiling feedback hooks.
- The test suite and implementation report claim 81/81 tests passing, including covariance PSD checks, projection gradients, depth gradients, rasterization, importance/tier classification, and CUDA forward kernels.
- The repository contains Replica and TUM RGB-D loaders, benchmark scripts, experiment scripts, and stored baseline/per-frame metrics.

2.2. Critical implementation gaps discovered

End-to-end real-time target is not met: The stored pipeline benchmark reports 403.54 ms mean frame time, 399.52 ms p50, 590.03 ms p95, and 2.48 FPS at 160x120 over 30 frames. This is far from the configured 16.6 ms frame budget. Therefore the system must currently be described as a research prototype, not a real-time system.

CUDA benchmark is not a full renderer benchmark: benchmarks/benchmark_render.py calls only `_C.preprocess_gaussians()` in `benchmark_cuda_rasterizer()`; it does not execute the complete CUDA binning + sorting + rasterization + depth + attribution pipeline. The 18,814 FPS number therefore cannot be presented as end-to-end CUDA rasterization FPS.

Normal-error term is currently inactive in the main pipeline: pipeline.py calls `update_statistics(... normal_errors=None)`. Thus the configured `normal_error` weight exists, but the normal component contributes zero during the current online importance computation.

Screen-area naming is misleading: The attribution code defines `screen_area` as total alpha-transmittance contribution weight, i.e. a contribution-weight proxy, not geometric projected ellipse area. The document should call this contribution mass/influence proxy unless the true projected area is computed.

Utility is still heuristic, not learned marginal utility: importance.py implements a weighted linear score with min-max normalization. It does not yet predict measured $\Delta\text{quality}/\Delta\text{compute}$. Therefore the “marginal utility” formulation is the research target, not an achieved result.

Budget scheduler controls only part of the declared budget: The scheduler uses an optimize-budget slice for Gaussian selection and a densification budget to cap additions, but render and memory budget shares are not yet independently enforced as scheduling decisions. The four-way allocation should therefore be presented as planned architecture rather than fully realized control.

New Gaussians receive an artificial importance initialization in pipeline.py: After densification, new Gaussians are padded with `importance=0.5` before scheduling if the estimator buffer has not produced values for them. This should be replaced by observation-aware initialization or a warm-up state so the scheduler does not inject an arbitrary prior.

Optimizer is recreated after every densification: pipeline.py calls `_setup_optimizer()` after adding Gaussians. This is simple and correct enough for a prototype, but optimizer-state resets can affect convergence and create avoidable overhead. A state-preserving parameter-group update is a later systems improvement.

Configuration drift exists: configs/default.yaml and OnlineReconstructionPipeline._default_config() disagree on important defaults such as SH degree and maximum Gaussian count. Experiments should use one canonical configuration source and record it with every result.

2.3. Correct interpretation of current experimental evidence

The current stored results are useful as engineering baselines, not yet as proof of the main research hypothesis. The strongest current evidence is that the core pipeline executes, gradients and CUDA primitives are testable, per-Gaussian attribution exists, and multiple scheduling policies can be compared. The next scientific step is to establish matched-budget comparisons and an oracle utility experiment.

Portfolio thesis: Designed and implemented an online RGB-D 3DGS research prototype with per-Gaussian error attribution, continuous importance scoring, and budget-aware scheduling; the current implementation is a validated research prototype, while end-to-end real-time performance and utility learning remain open research targets.

2. Research Question and Precise Problem Definition

2.1. Central research question

Làm thế nào để một online 3DGS system phân bổ một lượng compute hữu hạn vào đúng các Gaussian có marginal reconstruction utility cao nhất, thay vì tối ưu toàn bộ scene hoặc dùng một threshold cố định?

2.2. Formal problem

Given RGB-D frame F_t , Gaussian map G_t , and compute budget B_t :

choose action $A_t = \{\text{optimize}, \text{densify}, \text{prune}\}$

minimize $L(F_t, \text{Render}(G_t + \Delta G_t))$
subject to $\text{Cost}(A_t) \leq B_t$

where ΔG_t is restricted to the selected Gaussian subset.

Điểm quan trọng là budget không chỉ là số Gaussian. Cost thực tế có thể phụ thuộc vào số Gaussian, số pixel bị ảnh hưởng, projected footprint, kernel workload và memory movement. Vì vậy scheduler cần ưu tiên utility trên cost, không chỉ utility tuyệt đối.

2.3. Main hypothesis

Hypothesis H1: Với cùng compute budget, một scheduler sử dụng Gaussian-level utility và influence sẽ đạt reconstruction quality cao hơn các policy error-only, random hoặc fixed-frequency. H2: Khi budget thay đổi, utility-driven scheduling cho degradation graceful và tạo Pareto frontier tốt hơn full-optimization baseline xét trên quality, latency và memory.

4. Foundation: 3DGS and RTG-SLAM

3DGS biểu diễn scene bằng Gaussian primitives với vị trí μ , covariance Σ , opacity α và appearance thường dùng spherical harmonics. Covariance được parameterize bằng scale + rotation để duy trì tính hợp lệ. Differentiable projection biến Gaussian 3D thành elliptical footprint trong screen space.

$$\Sigma_{3D} = R S S^T R^T$$

$$\Sigma_{2D} \approx J W \Sigma_{3D} W^T J^T$$

Color rendering dùng alpha compositing front-to-back. Với RGB-D input, depth cung cấp geometry supervision trực tiếp. RTG-SLAM là baseline nền tảng của project: nó dùng compact Gaussian representation, depth rendering riêng với color rendering và selective online optimization.

RTG-SLAM đã force Gaussian về opaque/nearly-transparent roles, thêm Gaussian tại các vùng newly observed / color-error / depth-error và chỉ optimize unstable Gaussians. Do đó project phải vượt qua baseline này về compute allocation chứ không chỉ tái hiện các cơ chế của nó.

5. Research Gap and Positioning

4.1. What is already solved by the baseline

- Gaussian representation + differentiable rasterization.
- RGB-D online supervision và surface-aware depth rendering.
- Compact online mapping và selective optimization.
- Stable/unstable Gaussian selection và selective pixel rendering.

4.2. What this project adds

Problem	Baseline behavior	Proposed direction
Which Gaussian to optimize?	Binary stable/unstable or rule-based selection	Continuous utility / expected marginal gain
How much compute?	Fixed heuristic workload	Explicit latency/compute budget
What makes a Gaussian influential?	Mostly local error/visibility	Error × influence × uncertainty × temporal state
When to re-activate?	Rule/threshold driven	Temporal EMA + hysteresis + budget feedback
How to validate?	Quality + speed separately	Quality@Budget + Pareto frontier + oracle gap

Positioning statement: the novelty target is not “adaptive 3DGS” in the generic sense. The target is adaptive compute allocation for online RGB-D 3DGS at Gaussian level, measured under explicit latency and memory constraints.

6. Gaussian State and Utility Model

5.1. Per-Gaussian state

$s_i(t) = [e_{\text{rgb}}, e_{\text{depth}}, e_{\text{normal}}, \text{visibility}, \text{screen_area}, \text{influence}, \text{temporal_change}, \text{uncertainty}, \text{age}, \text{last_update}, \text{update_freq}, \text{gradient_norm}]$

State phải lưu cả thông tin tức thời và lịch sử. Một Gaussian có error cao trong một frame không nên bị xem giống một Gaussian có error cao ổn định trong hàng chục frame.

5.2. Influence

Influence đo mức độ Gaussian thực sự tác động tới image. Một approximation có thể dùng tổng contribution alpha-transmittance trên các pixel mà Gaussian phủ.

$$\text{Influence}_i \approx \sum_{p \in P_i} \alpha_{ip} T_{ip}$$

Influence giải quyết một thiếu sót của error-only scoring: hai Gaussian có cùng error nhưng Gaussian phủ 2000 pixel có expected impact khác Gaussian chỉ phủ 20 pixel.

5.3. Baseline utility score

$$I_i = w_1 E_{\text{rgb}} + w_2 E_{\text{depth}} + w_3 E_{\text{normal}} + w_4 \text{Visibility} + w_5 \text{TemporalChange} + w_6 \text{ScreenArea} + w_7 \text{Influence}$$

Weighted sum là baseline nghiên cứu, không được mặc định coi là final novelty.

5.4. Target utility: marginal quality gain

$$U_i = \Delta Q_i / (\text{Cost}_i + \epsilon)$$

ΔQ_i là quality improvement đo được nếu Gaussian i hoặc nhóm nhỏ chứa i được optimize. Đây là target có ý nghĩa nhất để kiểm chứng liệu một score có thực sự chọn đúng Gaussian hay không.

Giai đoạn đầu có thể dùng heuristic score để bootstrap. Sau đó thu thập training data dạng “Gaussian state → observed utility” và thử lightweight learned scorer. Neural scorer chỉ được triển khai sau khi baseline heuristic đã có kết quả rõ.

7. Separate Policies: Optimization, Densification, Pruning

Ba hành động có mục tiêu khác nhau và không nên dùng chung một importance score. Đây là thay đổi thiết kế quan trọng so với roadmap cũ.

Policy	Question	Main signals	Output
Optimization	Gaussian nào cần update?	error, influence, uncertainty, temporal drift, gradient	subset + frequency
Densification	Ở đâu cần thêm capacity?	error, gradient, geometric complexity, projected size	split/add candidates
Pruning	Gaussian nào có thể bỏ?	low contribution, low visibility, age, historical utility	evict candidates

6.1. Optimization tiers

Tier A: high utility / high uncertainty -> update now
 Tier B: medium utility -> update every N frames
 Tier C: stable / low utility -> freeze but keep rendering
 Tier D: persistent low contribution -> candidate for prune

Cần hysteresis để tránh trạng thái optimize/freeze liên tục. Ví dụ enter-active và leave-active dùng hai ngưỡng khác nhau.

6.2. Densification

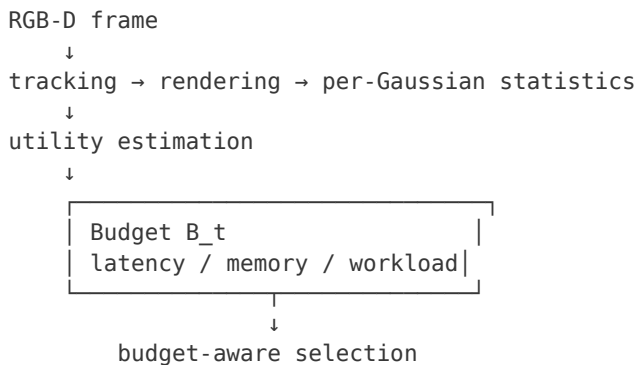
Densification nên ưu tiên vùng có high reconstruction error kết hợp với high gradient hoặc geometric complexity. Một hướng có ý nghĩa hình học là curvature-aware densification: mặt phẳng phẳng nhận ít Gaussian mới, còn edge/corner nhận nhiều hơn.

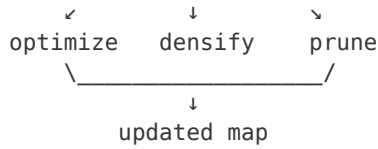
6.3. Pruning

Pruning phải dựa trên long-term contribution thay vì opacity đơn thuần. Gaussian low utility, low visibility và persistently low influence có thể bị prune; Gaussian có utility thấp hiện tại nhưng lịch sử quan trọng cần tránh xóa quá sớm.

8. Closed-Loop Budget-Aware Scheduler

Scheduler là phần nối ML và Systems. Model ước lượng utility; runtime quyết định action trong một ngân sách latency/memory/cụ thể.





7.1. Selection objective

$$\begin{aligned} & \text{maximize} \quad \sum_{i \in S} \text{Utility}_i \\ & \text{subject to} \quad \sum_{i \in S} \text{Cost}_i \leq B_t \end{aligned}$$

Baseline selection có thể dùng greedy utility/cost ratio hoặc top-k với cost approximation. Nếu profile cho thấy cost heterogeneity lớn, có thể chuyển sang knapsack-like selection.

7.2. Budget feedback

Budget có thể là 1/2/4/8 ms hoặc một latency target cố định. Khi runtime vượt target, scheduler giảm số update hoặc tăng activation threshold; khi runtime còn dư, scheduler có thể mở rộng subset. Đây là closed-loop adaptation, không phải threshold cố định.

7.3. Temporal statistics

$$\begin{aligned} \text{EMA}_t &= \beta \text{EMA}_{t-1} + (1-\beta) \text{error}_t \\ \text{Drift}_i &= \text{EMA}_{\text{recent}}(\text{error}_i) / (\text{EMA}_{\text{long}}(\text{error}_i) + \epsilon) \end{aligned}$$

Temporal drift được dùng để re-activate Gaussian và phát hiện vùng scene có thay đổi hoặc bị quan sát lại dưới điều kiện khác.

9. Learning Objective and Geometry

8.1. Baseline RGB-D objective

$$L_{\text{base}} = w_{\text{rgb}} L_{\text{rgb}} + w_{\text{depth}} L_{\text{depth}}$$

L1 có thể được dùng làm baseline. Sau đó mới ablate Charbonnier/Huber, normal consistency và temporal consistency.

8.2. Proposed objective

$$\begin{aligned} L_{\text{total}} &= L_{\text{rgbd}} + \lambda_{\text{geo}} L_{\text{geometry}} \\ &+ \lambda_{\text{temp}} L_{\text{temporal}} + \lambda_{\text{compact}} L_{\text{compact}} \end{aligned}$$

Không thêm mọi loss cùng lúc. Mỗi thành phần phải có ablation riêng để chứng minh contribution.

8.3. Uncertainty-aware depth

Depth sensor có thể có missing values, edge noise và depth confidence khác nhau. Một extension hợp lý là weighted depth loss:

$$L_{\text{depth}} = \sum_p w_{\text{depth}}(p) |D_{\text{gt}}(p) - D_{\text{render}}(p)|$$

Trong đó weight phụ thuộc reliability của depth observation. Đây là extension phụ, chỉ giữ nếu ablation cho thấy lợi ích rõ.

10. Empirical Status and Research Experiments

The repository already contains budget, policy, attribution, densification, normalization, and component-independence experiment scripts. The revised scientific protocol should treat these scripts as measurement infrastructure and separate current measured results from future target results.

10.1. Current measured prototype results

TUM RGB-D freiburg1_desk: 20-frame baseline in the stored report, starting from 16,008 Gaussians and ending at 23,690, with average depth L1 error 0.2356 m and PSNR 7.86 dB. These numbers are useful diagnostics but are not sufficient to establish reconstruction quality against a strong literature baseline.

End-to-end benchmark artifact: 30 frames at 160x120, mean 403.54 ms/frame, p50 399.52 ms, p95 590.03 ms, and 2.48 FPS. Therefore the current research question should be phrased around progress toward budget compliance, not an already-achieved real-time guarantee.

CUDA primitive benchmark artifact: the current benchmark measures preprocess kernel latency, not a complete rasterizer/render loop. Any paper/portfolio figure should label it as CUDA preprocessing unless the benchmark is expanded.

10.2. Required next experiments

- Matched-budget comparison: FULL vs RANDOM vs BINARY vs TOP_K vs BUDGET_AWARE at identical measured optimization time.
- Oracle utility experiment: temporarily optimize sampled Gaussian(s), measure actual Δ quality, and compare predicted ranking against the oracle with Spearman correlation, top-k overlap, and realized quality gain.
- Component ablation: error-only -> +depth -> +visibility -> +temporal -> +influence -> learned utility.
- Budget sweep: 1/2/4/8/16 ms target and report Quality@Budget plus p50/p95/p99 latency.
- Scale sweep: image resolution, Gaussian count, number of visible contributors, and attribution top-K.
- Failure analysis: flat surfaces, object edges, high-texture regions, sparse/invalid depth, and changing viewpoint.

11. Experimental Design - Required, Not Optional

11.1. Baselines

Baseline	Purpose
Full optimization	Quality upper bound under expensive compute
RTG-SLAM policy	Primary online RGB-D baseline
Random selection	Test whether score is better than random at same budget
Error-only	Test raw reconstruction error heuristic
Visibility-only	Test simple visibility heuristic
Error $\times$ influence	Strong non-learning heuristic
Proposed heuristic utility	Main non-learned method
Learned utility	Optional advanced version

11.2. Mandatory ablations

1. Binary stable/unstable vs continuous utility.
2. Error-only vs error + influence.
3. No temporal state vs temporal EMA.
4. No hysteresis vs hysteresis.
5. Fixed budget vs adaptive budget.
6. Uniform densification vs error/geometry-driven densification.
7. No LOD/render adaptation vs optional error-driven LOD.
8. Heuristic utility vs learned utility, only after sufficient training data exists.

11.3. Oracle experiment

For a sampled subset of Gaussians, temporarily measure actual quality improvement after optimization. This gives an oracle ranking. Compare the proposed score against oracle ranking using top-k overlap, Spearman correlation and realized quality gain. This experiment directly tests whether the scheduler is selecting the right Gaussians.

11.4. Budget stress test

Evaluate the same scene at multiple budgets, e.g. 1, 2, 4 and 8 ms mapping/optimization budget. Report Quality@Budget and latency percentiles. The main claim is not maximum FPS; it is better quality under the same budget.

12. Metrics and Success Criteria

Group	Metrics	Primary interpretation
Appearance	PSNR, SSIM, LPIPS	Novel-view / photometric quality
Geometry	Depth L1/AbsRel, accuracy, completion, Chamfer when GT exists	Surface quality
Tracking	ATE / trajectory error	Pose stability
Runtime	Mean, p50, p95, p99 frame latency	Real-time behavior
Efficiency	VRAM, peak memory, Gaussian count	Resource footprint
Adaptive quality	Quality@1/2/4/8 ms, Pareto frontier	Main research claim
Scheduler validity	Top-k overlap, rank correlation, oracle gap	Does utility predict actual gain?

Success criterion: at matched budget, proposed scheduling should either improve quality over strong baselines or maintain comparable quality with lower compute/memory. A speedup without matched quality is not sufficient evidence.

13. Dataset and Evaluation Strategy

Dataset	Role	Priority
Replica	Fast iteration, deterministic ablations	Primary development
TUM-RGBD	Tracking and real-world RGB-D behavior	Primary tracking check
ScanNet++	Large-scale geometry / scale stress	Secondary evaluation
Self-captured RGB-D	End-to-end demo and failure cases	Demo / robustness

Do not use self-captured data as the only evidence for scientific improvement. Public datasets should carry the quantitative claims; self-captured scenes should demonstrate generalization and runtime behavior.

14. AI Systems Layer - CUDA and Runtime

CUDA is an acceleration layer tied to a measured ML workload. The correct workflow is: implement correct baseline → profile → identify bottleneck → optimize one bottleneck → re-measure quality/latency/memory.

12.1. Suggested pipeline

Gaussian buffers

- preprocess / projection
- frustum + contribution culling
- tile binning
- compaction / prefix-sum
- depth sorting
- tile rasterization
- RGB/depth/transmission/statistics
- scheduler feedback

12.2. Memory layout

Prefer SoA or hybrid layout for position, scale/rotation, opacity, SH and metadata so kernels read only the fields they need. Measure memory bandwidth, cache behavior and VRAM use before changing layout.

12.3. Sorting

Use a mature radix sort such as CUB as baseline. Custom sorting is only justified if profiling shows sorting is a dominant bottleneck under the chosen workload.

12.4. Early termination

```
C += T * α * c
T *= (1 - α)
if T < ε: break
```

Early termination affects both speed and image quality, so every threshold change must be included in quality benchmarks.

12.5. Profiling checklist

- Kernel time and launch count
- SM utilization / occupancy
- Memory bandwidth and L2 behavior
- Warp divergence / branch efficiency
- VRAM and allocation frequency
- Frame-time p50/p95/p99
- Frame-time vs Gaussian count

15. Repository Architecture - Revised

```
adaptive_3dgs/
├── configs/
├── datasets/
├── gaussian/
│   ├── representation.py
│   ├── initialization.py
│   ├── covariance.py
│   └── state.py
├── rendering/
│   ├── projection.py
│   ├── rasterizer.py
│   ├── depth.py
│   └── statistics.py
├── optimization/
│   ├── losses.py
│   ├── optimizer.py
│   ├── densification.py
│   └── pruning.py
└── adaptive/
```

```

|
| — state.py
| — influence.py
| — importance.py
| — scorer.py
| — scheduler.py
| — thresholds.py
| — tracking/
| — cuda/
| — cpp/
| — experiments/
| — benchmarks/
| — viewer/
| — docs/

```

Điểm khác biệt quan trọng là adaptive/ trở thành module trung tâm. Điều này phản ánh trực tiếp research thesis thay vì làm repository giống một fork renderer.

16. Development Roadmap - New Order

Phase	Work	Exit criterion
0	Literature + equations	Notebook giải thích 3DGS/RTG-SLAM; unit tests cho projection/covariance/compositing
1	Static Gaussian renderer	Offline rendering đúng và benchmark được
2	RGB-D depth + losses	Gradient update đúng; depth stable
3	Online map growth	Gaussian addition + map update theo stream
4	RTG-SLAM reproduction	Baseline metrics và matching qualitative behavior
5	Instrumentation	Per-Gaussian state, influence, temporal stats
6	Utility baseline	Error/influence score + matched-budget experiments
7	Budget-aware scheduler	Optimization subset dưới explicit latency budget
8	Densification + pruning policies	Tách policy và ablation đầy đủ
9	Oracle + learned scorer	Utility prediction validated; learned model optional
10	CUDA acceleration	Profiler evidence + speedup at matched quality
11	Paper-style evaluation	Pareto curves, ablations, failure cases, reproducibility
12	Viewer/package	End-to-end demo; optional OpenGL polish

Scope cut rule: nếu thời gian hoặc complexity vượt kiểm soát, giữ Phase 0-8 và 11; bỏ Vulkan, memory tiering và viewer polish trước.

17. Failure Modes and How to Diagnose Them

Failure	Likely cause	Diagnostic
Quality drops at low budget	Utility score ignores influence or geometry	Compare oracle correlation and per-region quality
Scheduler oscillates	No temporal state/hysteresis	Plot active/inactive transitions per Gaussian
Gaussian count explodes	Densification too aggressive	Track count vs frame + prune rate
CUDA faster but quality worse	Aggressive culling / early termination	Matched-quality benchmark + visual diff
No gain vs RTG-SLAM	Novel score adds little over stable/unstable	Run error-only, influence-only and oracle gap
Learned scorer unstable	Insufficient utility labels / distribution shift	Evaluate on unseen scenes and use heuristic fallback

18. Final Research Definition

16.1. Core research contribution

A research framework for Gaussian-level utility estimation and compute allocation in online RGB-D 3DGS. The current repository implements heuristic importance and budget-aware selection; measured marginal utility prediction remains the central next research milestone.

16.2. Systems contribution

A C++/CUDA prototype that provides preprocessing, binning, sorting, rasterization, depth and per-Gaussian statistics kernels. The current end-to-end pipeline is not yet real-time; systems optimization must be driven by profiling and matched-quality measurements.

16.3. Evidence required for a credible claim

- Matched-budget comparison against RTG-SLAM and policy baselines, using measured compute rather than nominal budget estimates.
- Oracle experiment showing that proposed utility predicts realized quality gain.
- Ablations isolating influence, temporal state, hysteresis and budget adaptation.
- Quality@Budget and quality-latency-memory Pareto curves.
- Failure-case analysis on geometry edges, texture-heavy regions, sparse depth and varying scene scale.

19. References and Source Basis

- [1] Z. Peng, T. Shao, Y. Liu, J. Zhou, Y. Yang, J. Wang, K. Zhou. RTG-SLAM: Real-time 3D Reconstruction at Scale using Gaussian Splatting. SIGGRAPH 2024. arXiv:2404.19706. DOI: 10.1145/3641519.3657455.
- [2] B. Kerbl, G. Kopanas, T. Leimkuehler, G. Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. ACM TOG 42(4), 2023.
- [3] V. Yugay, Y. Li, T. Gevers, M. R. Oswald. Gaussian-SLAM: Photo-realistic Dense SLAM with Gaussian Splatting. arXiv:2312.10070, 2023.
- [4] E. Sandström, K. Tateno, M. Oechsle, M. Niemeyer, L. Van Gool, F. Tombari. Splat-SLAM: Globally Optimized RGB-only SLAM with 3D Gaussians. arXiv:2405.16544, 2024.
- [5] Z. Cao et al. RGBDS-SLAM: A RGB-D Semantic Dense SLAM Based on 3D Multi Level Pyramid Gaussian Splatting. arXiv:2412.01217, 2024.

Source distinction: the RTG-SLAM summary in the original project document is retained as baseline knowledge. Utility estimation, budget-aware scheduling, influence-based scoring, hysteresis and the revised closed-loop formulation are proposed extensions of this project, not claims made by RTG-SLAM.

20. One-Sentence Thesis

How can an online RGB-D 3DGS system spend a fixed GPU budget on the Gaussians that yield the highest marginal reconstruction benefit, while preserving quality and keeping latency predictable?

Repository audit source: Hieu475/adaptive_3dgs, main branch, inspected 2026-08-28. Key files: research/pipeline.py, research/importance.py, research/scheduler.py, research/attribution.py, benchmarks/benchmark_render.py, artifacts/implementation_report.md, artifacts/pipeline_benchmark.json, artifacts/benchmark_results.json, configs/default.yaml.